

线段树

线段树的本质是通过将题目要求的信息分治到树的端点上，然后通过懒处理进行维护

```
class segTree:
    def init(self,n,arr=None): #注意python和JS不一样，JS不需要写this，但是p
y的话必须将self显式地写在第一个
        self.n= n # 线段树大小
        self.tree= [(4n)] # 开一个4n大的数组存储查询目标值
        self.lazy_= [(4n)] # 懒标记

        if arr is not None:
            self.arr=[0]+arr #根节点编号从1开始
            self.build(1,1,n)

    def build(self,node,l,r):
        if l==r:
            # 叶子节点：直接赋值
            self.tree[node]=self.arr[l]
        else:
            # 分治赋值
            mid=(l+r)//2
            self.build(node*2 ,l ,mid)
            self.build(node*2+1,mid+1,r )
            #向上维护
            self.tree[node]=self.tree[node*2]+self.tree[node*2+1]

    def push(self,node,l,r):
        mid=(l+r)//2
        #分治
        ''' 注意懒标记处理优先级 '''
        if self.lazy_[node] is not None: # 必须用is not None，因为假值 0 也是有
```

效赋值

```
val=self.lazy_[node]
#处理值
self.tree[node*2]    #左子树: mid-l+1
self.tree[node*2+1]  #右子树: r-mid
#下放懒标记
self.lazy_[node*2]
self.lazy_[node*2+1]
''' 可能会对其他懒标记产生影响 '''
#清除当前节点懒标记
self.lazy_[node]
```

```
def update_(self,node,l,r,ql,qv,qr,val): # 对于不同的操作进行更新目标值
    # 1.完全覆盖: 直接在当前节点打懒标记, 不需要继续递归
    if ql<=l and qr>=r:
        self.tree[node]
        # 有时候根据优先级需要进行分支判断更新标记
        self.lazy_[node]

    else:
        # 2.部分覆盖: 需要继续递归, 所以 2.1先调用push初始化
        self.push(node,l,r)
        mid=(l+r)//2
        if ql<=mid: # 2.2通过判断确定更新方向
            self.update_ # node*2 ,l ,mid,ql,qv,qr,val
        if qr>mid:
            self.update_ # node*2+1,mid+1,r ,ql,qv,qr,val
        # 2.3向上维护: 更新当前节点的值 (一般求和就是直接+)
        self.tree[node]=self.tree[node*2]+self.tree[node*2+1]
```

```
def query(self,node,l,r,ql,qv,qr): # 查询目标值
    # 1.完全覆盖: 直接返回当前节点值
    if ql<=l and qr>=r:
        return self.tree[node]
    else:
        # 2.部分覆盖:
```

```

        self.push(node,l,r) # 2.1还是先调用push初始化
        mid=(l+r)//2
        res=0
        if ql<=mid:
            res+=self.query(node*2, l ,mid,ql,q,r,val)
        if qr>mid:
            res+=self.query(node*2+1,mid+1,r ,ql,q,r,val)
        return res

def solve():
    n=
    Tree=segTree(n)
    '''
    Tree.update( 1 , 1 , n , l , r , val )
        #根节点      输入范围 输入的更新值
    '''

if __name__=='__main__':
    solve()

```

MC0354

注意赋值的优先级比加减的优先级高

```

class segTree:
    def __init__(self,n,arr=None): #注意python和JS不一样，JS不需要写this，但是
        self.n= n # 线段树大小
        self.tree= [0]*( 4*n ) # 开一个4n大的数组存储查询目标值
        self.lazy_set=[None]*( 4*n ) # 懒标记
        self.lazy_add=[0]*(4*n)

        if arr is not None:
            self.arr=[0]+arr #根节点编号从1开始
            self.build(1,1,n)

```

```

def build(self,node,l,r):
    if l==r:
        # 叶子节点：直接赋值
        self.tree[node]=self.arr[l]
    else:
        # 分治赋值
        mid=(l+r)//2
        self.build(node*2 ,l ,mid)
        self.build(node*2+1,mid+1,r )
        #向上维护
        self.tree[node]=self.tree[node*2]+self.tree[node*2+1]

def push(self,node,l,r):
    mid=(l+r)//2
    #分治
    if self.lazy_set[node] is not None: # 必须用is not None，因为假值 0 也是有
        val=self.lazy_set[node]
        #处理值
        self.tree[node*2]=val*(mid-l+1)    #左子树：mid-l+1
        self.tree[node*2+1]=val*(r-mid)    #右子树：r-mid
        #下放懒标记
        self.lazy_set[node*2]=val
        self.lazy_set[node*2+1]=val
        self.lazy_add[node*2]=0
        self.lazy_add[node*2+1]=0
        #清除当前节点懒标记
        self.lazy_set[node]=None

    if self.lazy_add[node]:
        val=self.lazy_add[node]
        self.tree[node*2]+=val*(mid-l+1)
        self.tree[node*2+1]+=val*(r-mid)

        if self.lazy_set[node*2] is not None:
            self.lazy_set[node*2]+=val
        else:

```

```

        self.lazy_add[node*2]+=val

    if self.lazy_set[node*2+1] is not None:
        self.lazy_set[node*2+1]+=val
    else:
        self.lazy_add[node*2+1]+=val

    self.lazy_add[node]=0

def update_set(self,node,l,r,ql,q,r,val): # 对于不同的操作进行更新目标值
    # 1.完全覆盖：直接在当前节点打懒标记，不需要继续递归
    if ql<=l and q<=r:
        self.tree[node]=val*(r-l+1)
        # 有时候根据优先级需要进行分支判断更新
        self.lazy_set[node]=val
        self.lazy_add[node]=0 # 赋值操作会覆盖加减操作

    else:
        # 2.部分覆盖：需要继续递归，所以 2.1先调用push初始化
        self.push(node,l,r)
        mid=(l+r)//2
        if ql<=mid: # 2.2通过判断确定更新方向
            self.update_set(node*2,l,mid,ql,q,r,val) # node*2 ,l ,mid,ql,q,r,val
        if q>mid:
            self.update_set(node*2+1,mid+1,r,ql,q,r,val) # node*2+1,mid+1,r ,ql
        # 2.3向上维护：更新当前节点的值（一般求和就是直接+）
        self.tree[node]=self.tree[node*2]+self.tree[node*2+1]

def update_add(self,node,l,r,ql,q,r,val): # 对于不同的操作进行更新目标值
    # 1.完全覆盖：直接在当前节点打懒标记，不需要继续递归
    if ql<=l and q<=r:
        self.tree[node]+=val*(r-l+1)
        # 有时候根据优先级需要进行分支判断更新标记
        if self.lazy_set[node] is not None:
            self.lazy_set[node]+=val
        else:

```

```

        self.lazy_add[node] += val

    else:
        # 2.部分覆盖： 需要继续递归，所以 2.1先调用push初始化
        self.push(node,l,r)
        mid=(l+r)//2
        if ql<=mid: # 2.2通过判断确定更新方向
            self.update_add(node*2,l,mid,ql,q, val) # node*2 ,l ,mid,ql,q, val
        if qr>mid:
            self.update_add(node*2+1,mid+1,r,ql,q, val) # node*2+1,mid+1,r ,ql,q, val
        # 2.3向上维护： 更新当前节点的值（一般求和就是直接+）
        self.tree[node]=self.tree[node*2]+self.tree[node*2+1]

def query(self,node,l,r,ql,q,): # 查询目标值
    # 1.完全覆盖： 直接返回当前节点值
    if ql<=l and qr>=r:
        return self.tree[node]
    else:
        # 2.部分覆盖：
        self.push(node,l,r) # 2.1还是先调用push初始化
        mid=(l+r)//2
        res=0
        if ql<=mid:
            res+=self.query(node*2, l ,mid,ql,q, )
        if qr>mid:
            res+=self.query(node*2+1,mid+1,r ,ql,q, )
        return res

def solve():
    n,m=map(int,input().split())
    Tree=segTree(n)

    for _ in range(m):
        temp=list(map(int,input().split()))
        if temp[0]==1:
            l,r,val=temp[1],temp[2],temp[3]

```

```

        Tree.update_set(1,1,n,l,r,val)
    elif temp[0]==2:
        l,r,val=temp[1],temp[2],temp[3]
        Tree.update_add(1,1,n,l,r,val)
    else:
        l,r=temp[1],temp[2]
        print(Tree.query(1,1,n,l,r))

'''
Tree.update( 1 , 1 , n , l , r , val )
        #根节点      输入范围 输入的更新值
'''

if __name__=='__main__':
    solve()

```